PRÁCTICA · MODELOS DE LENGUAJE LOCALES

Tabú entre modelos locales

Documentación del proyecto y resultados del experimento

Pregunta experimental
¿Cómo afecta el número de palabras prohibidas (3, 5 o 7) al rendimiento de los modelos?

Resultado principal
El acierto cae del 27,5 % al 15,0 % al pasar de 3 a 7 prohibidas (McNemar p = 0,008, 120 pares).

Modelos
llama3.2:3b · qwen2.5:3b · gemma2:2b (Ollama, solo CPU)

Noviembre de 2026

Índice

1. Descripción general
2. Requisitos e instalación
3. Uso
4. Configuración
5. Arquitectura
6. Cómo funciona un turno
7. Detección de trampas
8. Tarjetas
9. Robustez y control de errores
10. Registro de partidas
11. Pruebas automáticas
12. Diseño experimental
13. Resultados obtenidos
14. Análisis y discusión
15. Limitaciones
16. Conclusiones

Parte I · Documentación del proyecto

1. Descripción general

`tabu_local` es un juego de Tabú en el que los tres jugadores son modelos de lenguaje de pesos abiertos ejecutados en local con Ollama, y un programa en Python hace de árbitro. Ninguna petición sale a Internet durante las partidas.

En cada turno, un modelo **descriptor** recibe una palabra secreta y una lista de palabras prohibidas y la describe. Otro modelo, el **adivinador**, intenta acertarla solo con esas pistas. Un **vigilante** (un tercer modelo) y las reglas del árbitro detectan trampas. Se juega por equipos y rondas: +1 por acierto y −1 por infracción.

El proyecto incluye:

- un motor de juego con árbitro determinista y aislamiento de información entre roles;
- detección de trampas en dos capas: reglas en Python y vigilante semántico;
- un mazo de 52 tarjetas en español validado automáticamente, y un generador de tarjetas nuevas;
- registro de cada partida en JSON e historial consultable;
- un experimento reproducible sobre el efecto de 3, 5 o 7 palabras prohibidas, con estadística y gráficos;
- una interfaz gráfica local en el navegador para la demostración en clase, y una versión de consola;
- 90 pruebas automáticas que no necesitan Ollama.

2. Requisitos e instalación

El proyecto funciona en Windows, macOS y Linux con Python 3.11 o superior y Ollama. Con los modelos por defecto (de 2 a 3 B parámetros) basta un equipo con 8 GB de RAM; con 16 GB, los tres modelos caben a la vez en memoria.

Componente	Versión o detalle
Python	3.11 o superior
Ollama	Versión reciente, desde ollama.com/download
Dependencias de Python	`requests`, `matplotlib`, `pytest` (en `requirements.txt`)
Disco	Unos 6 GB para los tres modelos

Pasos en Windows (PowerShell), desde la carpeta del proyecto:

1. Crear el entorno virtual: `python -m venv .venv`
2. Permitir scripts en esta ventana: `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`
3. Activar el entorno: `.venv\Scripts\Activate.ps1`. Aparece `(.venv)` delante del prompt.
4. Instalar las dependencias: `pip install -r requirements.txt`
5. Instalar Ollama y abrir la aplicación, o ejecutar `ollama serve`.
6. Descargar los modelos: `ollama pull llama3.2:3b`, `ollama pull qwen2.5:3b` y `ollama pull gemma2:2b`

7. Comprobar que todo está listo: `python main.py modelos`. Deben aparecer tres ✔.

Cada vez que se abra una terminal nueva basta con entrar en la carpeta y repetir los pasos 2 y 3. Una vez instalado, el proyecto funciona sin conexión a Internet.

3. Uso

La forma recomendada de jugar es la interfaz gráfica: `python main.py gui` abre el navegador en `http://127.0.0.1:8765`. El servidor solo escucha en el propio ordenador y no carga nada de Internet.

1. **Configuración:** comprueba Ollama y muestra los modelos instalados. Se eligen el descriptor y el adivinador de cada equipo (de 1 a 4 equipos), el vigilante (o ninguno), las rondas, las palabras prohibidas y los intentos.
2. **Precarga:** al empezar, carga los modelos en memoria y muestra el progreso de cada uno. La espera ocurre una sola vez.
3. **Partida:** muestra un panel por equipo y la tarjeta en el centro, con las fichas de intentos, un cronómetro y la conversación del turno. Las trampas aparecen tachadas dentro de la pista y en la tarjeta.
4. **Control:** *Siguiente turno* (o Enter) avanza en modo presentación, *Detener* corta la partida guardando lo jugado y la tecla F activa la pantalla completa.
5. **Final e historial:** una ventana anuncia al ganador, y la pestaña Historial muestra todas las partidas guardadas.

Comando	Qué hace
`python main.py`	Menú interactivo
`python main.py gui`	Interfaz gráfica (`--puerto`, `--no-abrir`)
`python main.py modelos`	Comprueba Ollama y los modelos de la configuración
`python main.py demo`	Partida guiada en consola, con pausa entre turnos
`python main.py jugar`	Partida (`--rondas`, `--prohibidas`, `--intentos`, `--sin-vigilante`, `--sin-pausa`)
`python main.py historial [ID]`	Lista de partidas o detalle de una
`python main.py experimento`	Experimento 3/5/7 (`--tarjetas`, `--repeticiones`, `--reanudar`, `--si`)
`python main.py analizar CARPETA`	Recalcula las estadísticas y los gráficos de un experimento
`python main.py tarjetas validar`	Revisa el mazo
`python main.py tarjetas generar -n 5`	Genera tarjetas nuevas con un modelo local

Modo simulado: la opción `--simulado` sustituye los modelos por un simulador aleatorio. Sirve para probar el programa sin Ollama; sus resultados son sintéticos y quedan marcados como tales.

4. Configuración

Todo se configura en `config.json`, sin tocar el código. El programa rechaza claves desconocidas y valores fuera de rango con un mensaje claro.

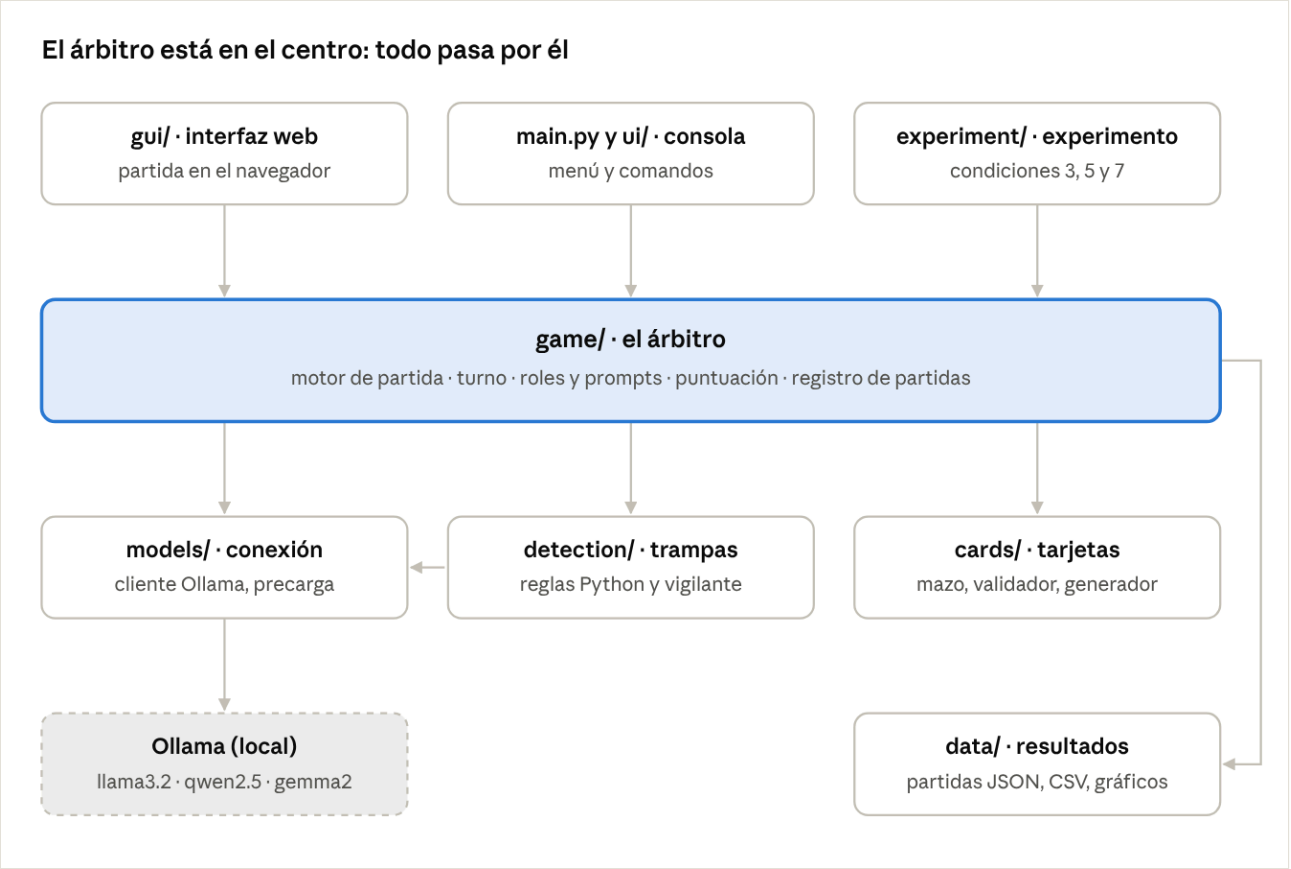
Equipo	Describe	Adivina
Equipo Azul	`llama3.2:3b`	`qwen2.5:3b`
Equipo Rojo	`qwen2.5:3b`	`llama3.2:3b`
Vigilante (común)	`gemma2:2b`	—

Sección	Clave	Defecto	Para qué sirve
`ollama`	`num_gpu`	0	0 = todo en CPU, para que los modelos quepan a la vez en la RAM
`ollama`	`timeout_s`	90	Tiempo máximo de respuesta de un modelo (s)
`ollama`	`temperature` / `seed`	0,7 / 42	Muestreo y reproducibilidad
`ollama`	`max_tokens`	200	Límite de tokens por respuesta
`juego`	`rondas`	3	En cada ronda juega un turno cada equipo
`juego`	`intentos_por_tarjeta`	3	Respuestas del adivinador por tarjeta
`juego`	`num_prohibidas`	5	Prohibidas por tarjeta (1–7; el juego normal usa 3–5)
`juego`	`puntos_acierto` / `penalizacion_infraccion`	1 / 1	Puntuación
`juego`	`fin_turno_por_infraccion`	true	Una trampa termina el turno
`juego`	`max_palabras_descripcion`	40	Longitud máxima de una pista
`juego`	`vigilante_modo`	penaliza	`penaliza` o `consultivo` (solo registra)
`juego`	`umbral_confianza_vigilante`	0,8	Confianza mínima para que cuente una sospecha
`experimento`	`condiciones`	3, 5, 7	Palabras prohibidas en cada condición
`experimento`	`num_tarjetas` / `repeticiones`	30 / 2	Tamaño del experimento

Por seguridad, el cliente solo acepta hosts locales y rechaza los modelos `*-cloud` de Ollama, que se ejecutan en servidores remotos.

5. Arquitectura

El proyecto es modular y el árbitro (`game/`) está en el centro. La interfaz web, la consola y el experimento usan exactamente el mismo motor, y el motor usa los modelos, las reglas anti-trampa y las tarjetas.



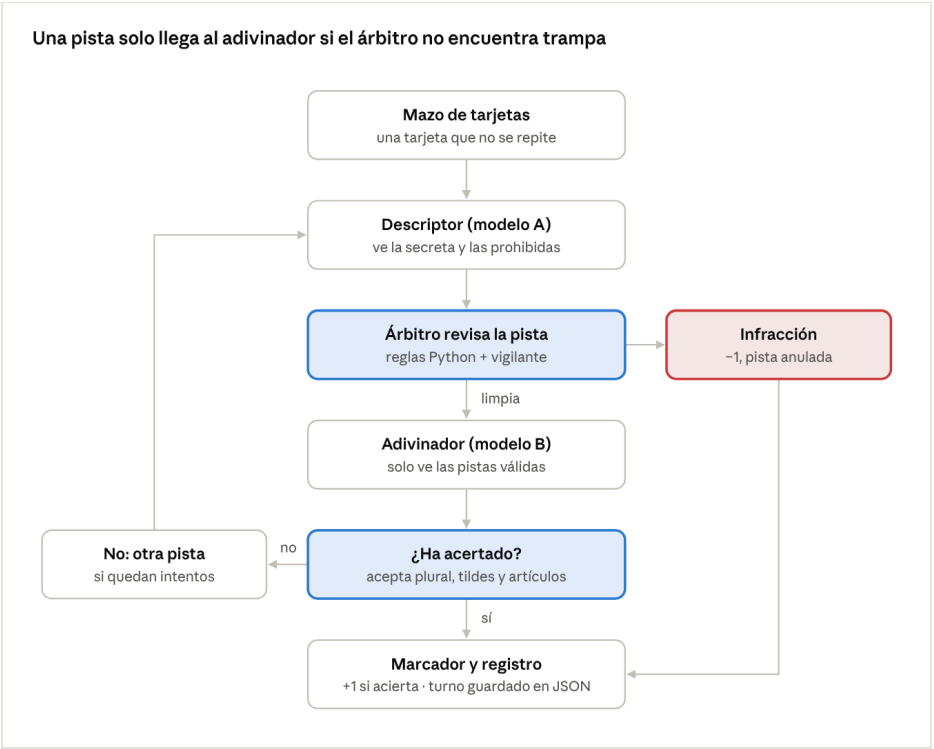
Arquitectura: interfaces, árbitro y módulos.

Principios de diseño:

1. Los modelos nunca deciden el resultado: producen texto y el árbitro, en Python, decide.
2. Cada rol recibe solo su información, lo que garantizan la firma de las funciones y las pruebas.
3. Nada bloquea la partida: cada fallo se registra y se recupera de forma controlada.
4. Toda la configuración vive en `config.json`.
5. Solo inferencia local: se rechazan los hosts remotos y los modelos `*-cloud`.

6. Cómo funciona un turno

Un turno es un equipo intentando adivinar una tarjeta con un número limitado de intentos. El árbitro decide todo; los modelos solo producen texto.



Flujo de un turno, del mazo al registro.

Rol	Ve	No ve
Descriptor	Palabra secreta, prohibidas, sus pistas anteriores y las respuestas fallidas del adivinador	Marcador y otros equipos
Adivinador	Pistas válidas y sus propias respuestas fallidas	Palabra secreta, prohibidas y pistas anuladas
Vigilante	La pista que revisa, la secreta y las prohibidas	Equipos, marcador y respuestas del adivinador

La función del adivinador no tiene ningún parámetro por el que pueda recibir la palabra secreta, y una prueba automática revisa todos sus prompts. Un acierto acepta mayúsculas, tildes, artículos, plural y espacios («arco iris» = «arcoíris»), pero no cambios de género ni diminutivos.

7. Detección de trampas

Las trampas se detectan en dos capas. Las reglas deterministas en Python deciden lo comprobable, y el vigilante semántico solo opina sobre lo que no se puede comprobar de forma literal. Una pista con una infracción nunca llega al adivinador.

Capa 1: reglas en Python. Antes de comparar, el texto se normaliza: minúsculas, sin tildes y sin signos de puntuación.

Tipo	Ejemplo detectado	Cómo
Exacta	«perro»	Palabra igual
Morfológica	«perros», «perra», «perrito»	Plurales, género, diminutivos y aumentativos
Derivada	«perrera», «ladra»	Comparte la raíz (4 letras o más)
Compuesta	«paraguas» contiene «agua»	El objetivo aparece dentro de otra palabra
Frase	«echar de menos»	Prohibidas de varias palabras
Deletreo	«p-e-r-r-o»	Letras sueltas seguidas
Traducción	«dog», «chien»	Variantes registradas en cada tarjeta

Capa 2: vigilante semántico. Solo se consulta si las reglas no encuentran nada. Busca sinónimos evidentes, traducciones no registradas, faltas de ortografía intencionadas y pistas de letras o rimas. Responde con un JSON fijo (`infraccion`, `palabra_detectada`, `tipo_infraccion`, `justificacion`, `nivel_confianza`). El árbitro solo acepta su sospecha si cumple tres condiciones:

- el JSON es válido;
- la confianza alcanza el umbral;
- el fragmento citado aparece literalmente en la pista.

8. Tarjetas

El mazo base tiene 52 tarjetas en español, escritas a mano, de 14 categorías y 3 dificultades. Cada tarjeta tiene:

- 5 prohibidas principales, ordenadas por relevancia;
- 2 prohibidas extra, que solo usa la condición de 7;
- traducciones de la palabra secreta.

El validador rechaza tarjetas incompletas o con prohibidas repetidas, prohibidas que son la propia secreta, palabras vacías o secretas repetidas en el mazo. El mazo nunca repite una tarjeta en la misma sesión. Un generador opcional crea tarjetas nuevas con un modelo local y las valida antes de guardarlas aparte.

9. Robustez y control de errores

Situación	Qué hace el programa
Respuesta vacía, demasiado corta o repetida	La descarta y pide otra con un prompt más estricto
El modelo habla de las reglas o de su rol	La descarta como inválida y pide otra
Pista demasiado larga	La recorta por la última frase completa
Respuesta sin formato JSON	Extrae la palabra del texto libre
El adivinador repite una respuesta fallida	Le pide otra distinta
Tiempo de espera agotado	Lo registra y reintenta
Modelo no instalado	Avisa antes de empezar, con la orden `ollama pull` necesaria
Ollama cae durante la partida	Detiene la partida de forma controlada y guarda lo jugado
Vigilante sin JSON válido o sin pruebas	No penaliza y lo registra

10. Registro de partidas

Cada partida se guarda en `data/games/<partida_id>.json` tras cada turno. El fichero contiene la configuración, el estado final, el marcador y un registro por turno con:

- los modelos que jugaron;
- la palabra secreta y las prohibidas;
- las pistas entregadas y las rechazadas, con su motivo;
- las respuestas del adivinador, el resultado y la puntuación;
- las infracciones y los veredictos del vigilante;
- los tiempos y los errores.

El historial se consulta en la interfaz gráfica o con `python main.py historial`.

11. Pruebas automáticas

Las 90 pruebas se ejecutan con `python -m pytest` y ninguna necesita Ollama: usan clientes falsos que devuelven respuestas guionizadas.

Fichero	Qué comprueba
`test_detection.py`	Normalización, cada tipo de trampa, JSON y aislamiento del vigilante
`test_cards.py`	Mazo válido, validador, mazo sin repeticiones y generador
`test_game.py`	Aciertos, infracciones, aislamiento del adivinador, recuperación de errores y registro
`test_scoring.py`	Puntuación, marcador y desempates
`test_experiment.py`	Estadística, semillas por condición, reanudación, McNemar y auditoría del vigilante
`test_ollama_client.py`	Errores de red, solo hosts locales, opción de CPU y precarga
`test_gui.py`	API de la interfaz: partida, precarga, pausas y detener

Parte II · Experimento y resultados

12. Diseño experimental

- **Pregunta:** ¿cómo afecta el número de palabras prohibidas al rendimiento?
- **Variable independiente:** número de prohibidas. Condición A = 3, B = 5, C = 7.
- Las condiciones son **anidadas**: las 3 de A están dentro de las 5 de B, y estas dentro de las 7 de C. Así solo cambia la cantidad de restricciones, no qué palabras se prohíben.
- **Diseño intra-sujetos:**
- Las mismas tarjetas (por defecto 30, con selección fija por semilla), los mismos equipos, el mismo vigilante y la misma configuración en las tres condiciones.
- La misma **semilla de muestreo** por (repetición, tarjeta, equipo) en las tres condiciones.
- **Control de orden:** para cada (repetición, tarjeta, equipo) se juegan las tres condiciones seguidas, en orden aleatorio con semilla. Así no se confunden con la condición posibles derivas del sistema (caché, temperatura del equipo, otros procesos).
- **Repeticiones:** 2 por defecto, con semillas distintas, para estimar la variabilidad del muestreo.
- **Variables dependientes:**
- % de acierto;
- infracciones por turno (automáticas y semánticas por separado);
- descripciones inválidas;
- tiempo medio de respuesta del descriptor y del adivinador;
- intentos necesarios (solo en turnos acertados);
- puntuación media por ronda (= por turno).
- **Análisis:**
- medias con desviación típica e IQR;
- IC 95 %: de Wilson para proporciones y bootstrap para medias;
- **diferencias pareadas** entre condiciones (mismo turno en dos condiciones) con IC 95 % bootstrap.
- El análisis pareado elimina la variabilidad debida a la dificultad de cada tarjeta.
- **Hipótesis a contrastar:** más prohibidas → más infracciones y menos aciertos. Es una hipótesis, no un resultado.

13. Resultados obtenidos

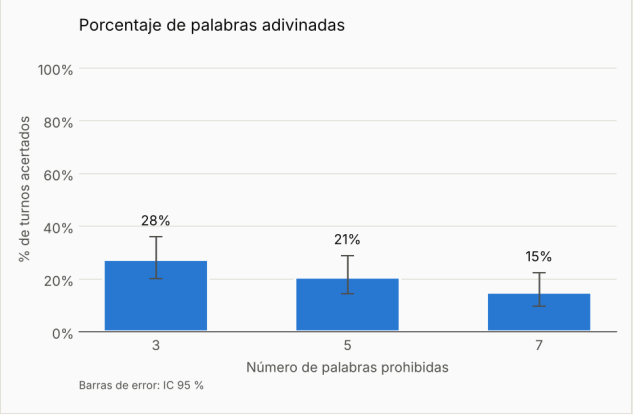
Ejecución. Experimento `exp_20261106_114534`, ejecutado el 6 de noviembre de 2026 en un portátil con Windows, 16 GB de RAM y los modelos solo en CPU (`num_gpu: 0`). Se jugaron 360 turnos: 30 tarjetas × 2 equipos × 3 condiciones × 2 repeticiones, es decir, 120 turnos por condición. Duró 2 horas. El vigilante (`gemma2:2b`) estuvo en modo `penaliza`.

Equipo	Describe	Adivina
Azul	`llama3.2:3b`	`qwen2.5:3b`
Rojo	`qwen2.5:3b`	`llama3.2:3b`

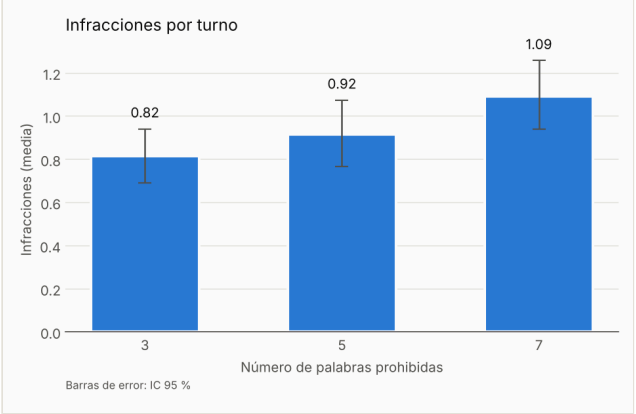
13.1 Resultados por condición

Condición	% acierto [IC 95 %]	Infracciones/turno [IC 95 %]	Inválidas/turno	Intentos (aciertos)	Puntuación/turno [IC 95 %]	t. descriptor (s)
A (3 prohibidas)	**27,5 %** [20,3–36,1]	0,82 [0,69–0,94]	0,03	1,39 ± 0,61	−0,54 [−0,73, −0,34]	4,1 ± 2,0
B (5 prohibidas)	**20,8 %** [14,5–29,0]	0,92 [0,77–1,08]	0,00	1,52 ± 0,65	−0,71 [−0,91, −0,51]	4,5 ± 2,6
C (7 prohibidas)	**15,0 %** [9,7–22,5]	1,09 [0,94–1,26]	0,02	1,17 ± 0,38	−0,94 [−1,14, −0,75]	4,3 ± 1,9

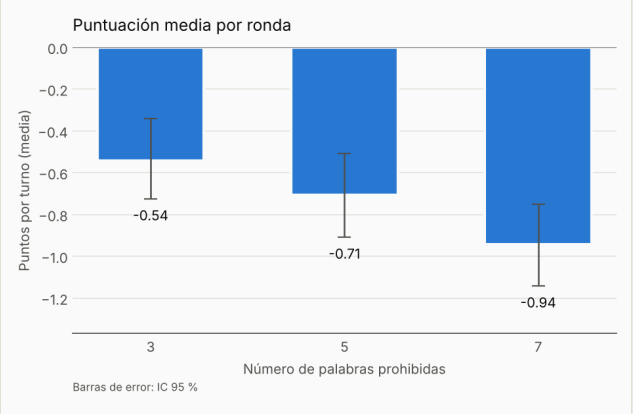
Fuente: `resumen_condiciones.csv`.



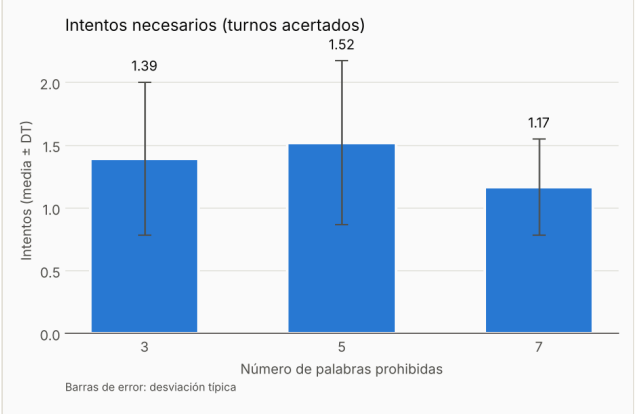
Porcentaje de palabras adivinadas (IC 95 %).



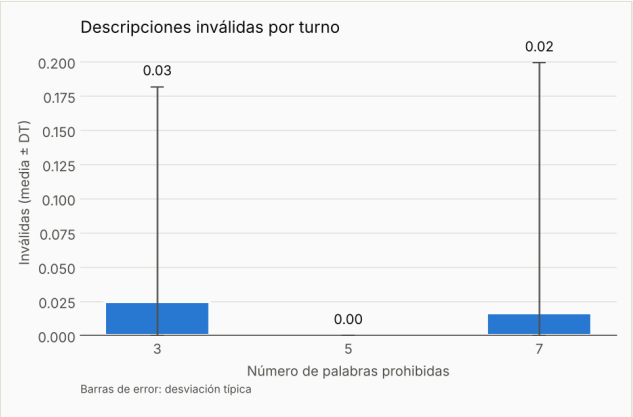
Infracciones por turno (IC 95 %).



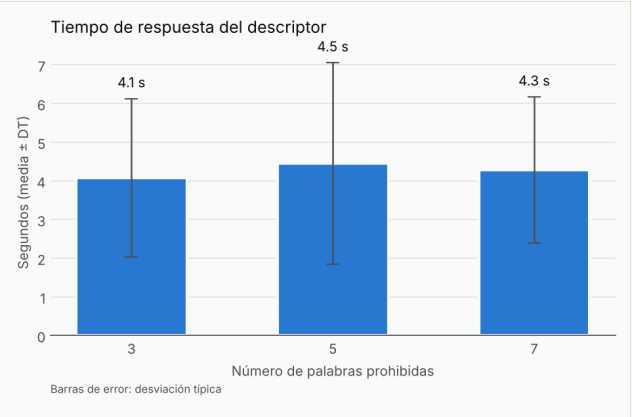
Puntuación media por turno (IC 95 %).



Intentos en los turnos acertados (media ± DT).



Descripciones inválidas por turno (media ± DT).



Tiempo de respuesta del descriptor (media ± DT).

13.2 Comparaciones pareadas

Cada turno se compara consigo mismo en otra condición: misma tarjeta, equipo y repetición (120 pares).

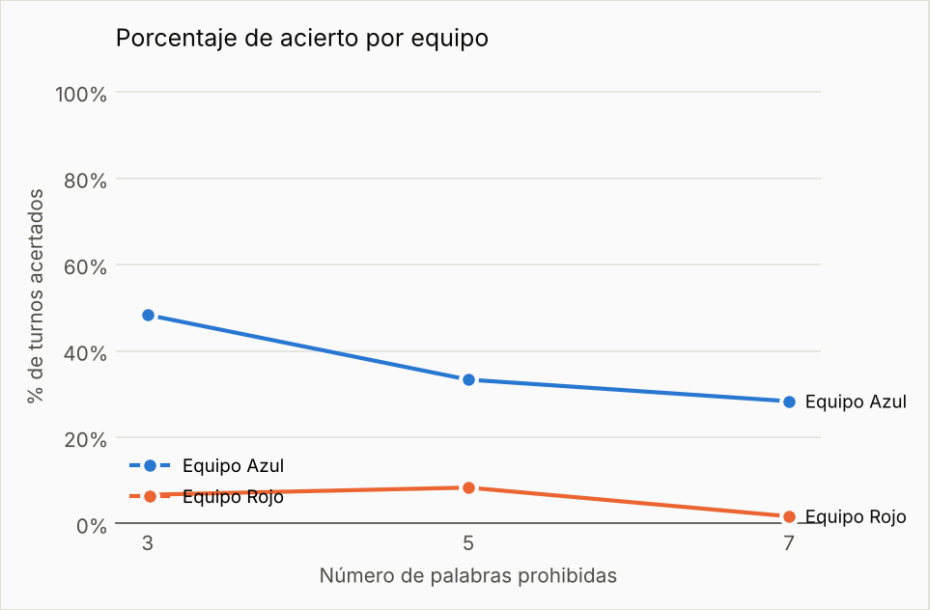
Comparación	Δ acierto [IC 95 %]	McNemar (p)	Δ infracciones [IC 95 %]	Δ puntuación [IC 95 %]
3 → 5	−6,7 pts [−15,8, +1,7]	0,20	+0,10 [−0,04, +0,24]	−0,17 [−0,38, +0,03]
3 → 7	**−12,5 pts** [−20,8, −4,2]	**0,008**	**+0,28** [+0,13, +0,43]	**−0,40** [−0,60, −0,20]
5 → 7	−5,8 pts [−14,2, +2,5]	0,25	**+0,18** [+0,03, +0,33]	**−0,23** [−0,44, −0,04]

En negrita, las diferencias cuyo intervalo excluye el cero. Fuentes: `comparaciones.csv` y `mcnemar.csv`. En la comparación 3 → 7 hay 22 tarjetas que se acertaron con 3 prohibidas y se fallaron con 7, frente a solo 7 en sentido contrario.

13.3 Resultados por equipo

Condición	Azul: % acierto	Rojo: % acierto	Azul: infracc./turno	Rojo: infracc./turno
3	48,3 % [36,2–60,7]	6,7 % [2,6–15,9]	0,50	1,13
5	33,3 % [22,7–45,9]	8,3 % [3,6–18,1]	0,57	1,27
7	28,3 % [18,5–40,8]	1,7 % [0,3–8,9]	0,63	1,55

Fuente: `resumen_equipos.csv`.



Porcentaje de acierto por equipo y condición.

13.4 Origen de las infracciones

De las 339 infracciones, 320 las detectaron las reglas en Python y 19 el vigilante.

	Equipo Azul	Equipo Rojo
Turnos en los que el descriptor escribió la **palabra secreta**	35 de 180 (19 %)	**116 de 180 (64 %)**
Infracciones por usar una **palabra prohibida**	62	107

Condición	Turnos con palabra prohibida	Turnos con la palabra secreta
3	36	50
5	42	53
7	63	48

Por tipo de detección, 248 infracciones fueron coincidencias exactas, 51 morfológicas (plurales, género, diminutivos) y 21 por derivación.

13.5 Acierto en los turnos sin infracción

Con esta tabla se separan los dos mecanismos posibles: hacer trampa frente a describir peor.

Condición	Azul: turnos limpios	Azul: % acierto [IC 95 %]
3	33	87,9 % [72,7–95,2]
5	31	64,5 % [47,0–78,9]
7	28	60,7 % [42,4–76,4]

El Equipo Rojo tuvo muy pocos turnos limpios (6, 8 y 3), así que sus porcentajes no son interpretables. Fuente: `acierto_sin_infraccion.csv`.

13.6 Comportamiento del vigilante

Fuente: `auditoria_vigilante.json`.

- Emitió 269 veredictos, todos con JSON válido.
- Acusó de infracción en 196 de ellos (**73 %**), casi siempre con una confianza de 0,9 o 1,0.
- El árbitro descartó **177 de las 196 acusaciones (90 %)**, porque el fragmento citado no aparecía en la pista. Casi siempre afirmaba que la palabra secreta estaba escrita, y no lo estaba.
- Se aplicaron 19 penalizaciones semánticas. En la revisión manual, solo una es defendible: «La primera sílaba es "qui-"», en la tarjeta *médico*, que es una pista de letras. Las otras 18 son descripciones legítimas, por ejemplo «almacenar alimentos frescos» (nevera), «tiene camillas y monitores» (hospital) o «es lo que transporta a gente a otra ciudad» (tren).

Análisis de sensibilidad. Se repitió la comparación excluyendo los 16 tríos de turnos afectados por una penalización semántica (104 pares). El acierto queda en 28,8 %, 23,1 % y 16,3 %, y la diferencia 3 → 7 sigue siendo significativa (McNemar p = 0,019). Las penalizaciones del vigilante no cambian las conclusiones.

14. Análisis y discusión

1. Más palabras prohibidas reducen el rendimiento. El acierto baja de forma monótona: 27,5 %, 20,8 % y 15,0 %. La caída de 3 a 7 (−12,5 puntos) es estadísticamente consistente según el IC pareado y McNemar (p = 0,008). Los pasos intermedios (3 → 5 y 5 → 7) van en la misma dirección, pero con 120 pares no alcanzan a distinguirse del azar. Lo mismo ocurre con la puntuación (−0,54, −0,71, −0,94) y con las infracciones (0,82, 0,92, 1,09). La hipótesis se confirma para la diferencia grande, no para las pequeñas.

2. El efecto llega por dos vías.

- **Más trampas involuntarias.** Los turnos con una palabra prohibida suben de 36 a 42 y a 63. En cambio, los turnos con la palabra secreta no varían (50, 53, 48), como cabía esperar, porque la secreta está prohibida en todas las condiciones. Por tanto, el aumento de infracciones se debe a las palabras añadidas a la lista.
- **Peores descripciones, incluso sin trampas.** En el Equipo Azul, cuando el descriptor respeta las reglas, el acierto cae del 87,9 % (3 prohibidas) al 64,5 % y al 60,7 %. Al prohibir las palabras más útiles, las pistas que quedan son menos informativas.

3. Las prohibidas más relevantes son las que más se usan. Las infracciones por posición en la lista fueron 61, 46, 19, 20, 8, 8 y 7. Las dos primeras palabras, las más ligadas a la secreta, concentran la mayoría. Las extra (posiciones 6 y 7) apenas se usaron, lo que explica por qué el paso de 5 a 7 tiene un efecto menor.

4. La pareja de modelos importa más que el número de prohibidas. La diferencia entre equipos (48 % frente a 7 % con 3 prohibidas) es mucho mayor que la diferencia entre condiciones. `qwen2.5:3b` como descriptor escribió la palabra secreta en el 64 % de sus turnos («Un helicóptero tiene dos hélices…»): no respeta la regla principal del juego. Con los mismos modelos y los papeles cruzados, `llama3.2:3b` lo hace en el 19 %. Por eso el efecto de la condición se observa sobre todo en el Equipo Azul, ya que el Rojo está cerca del 0 % en todas.

5. Un modelo pequeño no sirve como juez. `gemma2:2b` acusó en el 73 % de los casos y, en 9 de cada 10 acusaciones, inventó la prueba. Sin la regla de exigir que el fragmento citado aparezca en la pista, habría penalizado unas 196 veces en lugar de 19, y el experimento habría medido sobre todo el ruido del juez. Es un argumento a favor del diseño híbrido: las reglas deterministas deciden lo comprobable y el modelo solo opina, siempre con pruebas.

6. Tiempos. El tiempo de respuesta del descriptor no cambia con la condición (4,1, 4,5 y 4,3 s): un prompt algo más largo no afecta de forma apreciable. En cambio, la duración de los turnos depende sobre todo del equipo. Los del Azul duran más porque llegan más a menudo al adivinador y al vigilante, mientras que los del Rojo suelen cortarse en la primera pista.

15. Limitaciones

- **Validez externa limitada:** los resultados son de estos tres modelos de 2–3 B, estos prompts y este mazo. Con modelos mayores el efecto podría ser distinto.
- **Tamaño muestral:** 120 pares por comparación permiten detectar la diferencia 3 → 7 (unos 12 puntos), pero no los pasos de 2 en 2 (unos 6 puntos). Para distinguirlos harían falta varios cientos de pares.
- **Equipos desequilibrados:** el Equipo Rojo apenas acierta, de modo que el efecto de la condición se estima casi solo con el Equipo Azul.
- **Una infracción termina el turno:** con esta regla, las trampas y las descripciones peores se entrelazan. El apartado 7.5 los separa en parte; un experimento con `fin_turno_por_infraccion: false` lo haría mejor.
- **Vigilante en modo penaliza:** 19 penalizaciones (5,6 % de las infracciones), casi todas falsas, afectaron a 16 tríos de turnos. El análisis de sensibilidad muestra que no cambian las conclusiones, pero en una repetición conviene usar el modo `consultivo`.
- **Detector automático:** en la revisión aparecieron pocos falsos positivos dudosos, como «suelen» detectado como derivado de «suelo», o «marina» de «mar».
- **Revisión manual parcial:** la clasificación de las 19 infracciones semánticas la hizo una sola persona; no se revisaron todos los veredictos.
- **Determinismo parcial:** otra máquina u otra versión de Ollama pueden dar respuestas y tiempos distintos.
- **Sin modelo estadístico completo:** se usan IC, comparaciones pareadas y McNemar. Una regresión logística de efectos mixtos, con la tarjeta y el equipo como efectos aleatorios, sería más precisa.

16. Conclusiones

1. **Ampliar la lista de prohibidas empeora el rendimiento:** de 3 a 7 palabras, el acierto cae del 27,5 % al 15,0 % (McNemar p = 0,008) y las infracciones suben de 0,82 a 1,09 por turno. Los pasos intermedios siguen la misma tendencia sin alcanzar significación con esta muestra.
2. **El empeoramiento llega por dos vías:** más palabras prohibidas usadas sin querer y pistas menos informativas incluso cuando se respetan las reglas. En el mejor equipo, el acierto de los turnos limpios baja del 88 % al 61 %.
3. **La elección del modelo descriptor pesa más que la regla:** `qwen2.5:3b` escribe la palabra secreta en dos de cada tres turnos, y `llama3.2:3b` en uno de cada cinco.
4. **Un modelo de 2 B no es un juez fiable:** el diseño híbrido (reglas deterministas más vigilante obligado a aportar pruebas) evitó el 90 % de sus acusaciones falsas.

Además, el proyecto ofrece un entorno completo y reproducible:

- un árbitro determinista que aísla la información de cada rol;
- una detección de trampas en dos capas;

- un mazo validado;
- registros completos;
- un experimento intra-sujetos con análisis pareado;
- 90 pruebas automáticas;
- una interfaz gráfica para la demostración en clase.